

SIMATS ENGINEERING

ASSESSMENT TOOL 3

**COURSE NAME: CRYPTOGRAPHY AND NETWORK
SECURITY COURSE CODE: CSA51**

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4, BL5)

Debugging & Optimisation Task: 10%

QUESTIONS: Total Marks: 50 Annexure A

Code of Conduct:

I Eddu Suchier/192524070 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

**Annexure A
Debugging & Optimisation Task**

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)

A. Identify and fix logical and implementation errors in key scheduling and permutation steps

The inconsistent ciphertext for identical plaintext and key indicates errors in the DES/3DES implementation rather than the algorithm itself, since DES is deterministic. The most common issue is incorrect key scheduling, where the 56-bit key is not properly generated using the PC-1 permutation, split into two 28-bit halves, and circularly shifted according to the DES shift schedule before applying the PC-2 permutation to create the 16 round keys. Another possible error is incorrect implementation of the Initial Permutation (IP), Final Permutation (FP), Expansion (E), P-box, or S-box substitution tables. These permutation tables must be applied exactly as defined in the DES standard. For 3DES, ensure that encryption follows the correct sequence (Encrypt–Decrypt–Encrypt or EDE mode) using three valid keys. Remove any random key generation inside the encryption function and generate the round keys only once for each key. Adding unit tests using known DES test vectors can help verify that every permutation and round key is produced correctly.

B. Optimize the code to reduce execution time for large input datasets (≥10 MB)

To improve performance for large datasets, avoid recalculating round keys for every block of plaintext. Instead, generate the 16 round keys once and reuse them throughout the encryption process. Replace nested loops in permutation functions with precomputed lookup tables or list comprehensions to reduce execution time. Process the input data block-by-block (64-bit blocks) using efficient data structures such as bytes or bytearray instead of repeatedly manipulating strings. Minimize unnecessary binary-to-string conversions and avoid repeated memory allocations within encryption rounds. For very large files, implement buffered file reading and writing so the entire dataset does not need to be loaded into memory at once. These optimizations significantly improve execution speed and memory efficiency without affecting the correctness of the encryption.

C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance

Replacing DES or 3DES with a modern encryption algorithm such as AES improves both security and performance. DES uses only a 56-bit key, making it vulnerable to brute-force attacks, while 3DES provides stronger security by applying DES three times but suffers from slow execution and has a relatively small block size of 64 bits. AES supports 128-bit blocks with key sizes of 128, 192, or 256 bits, providing much stronger protection against modern attacks. In addition, AES is optimized in both software and hardware, allowing it to encrypt large amounts of data much faster than 3DES. As a result, AES is the current industry standard for secure and efficient symmetric encryption and is recommended for new applications over DES or 3DES.

2. Performance Analysis of Block Cipher Modes of Operation

A. Debug incorrect padding and IV handling causing decryption failures

Decryption failures in AES block cipher modes are commonly caused by incorrect padding or improper Initialization Vector (IV) handling. For ECB and CBC modes, the plaintext must be padded correctly (e.g., using PKCS#7 padding) before encryption, and the padding must be removed only after successful decryption. The IV should be exactly 16 bytes for AES and must be unique for each encryption session. In CBC, CFB, and OFB modes, the same IV used during encryption must also be provided during decryption; generating a new IV during decryption will produce incorrect plaintext. Additionally, ensure that the IV is not modified during processing and is transmitted or stored securely alongside the ciphertext. Verifying padding, IV size, and IV consistency eliminates most decryption errors.

B. Refactor the code to minimize redundant computations and memory usage

The implementation can be optimized by removing repeated computations and improving memory management. Initialize the AES key schedule only once and reuse it for all data blocks instead of regenerating it repeatedly. Process data in fixed-size blocks or streams rather than loading the entire file into memory, which reduces memory consumption for large datasets. Avoid unnecessary copying of arrays or repeated binary/string conversions by using bytes or bytearray objects. Reuse temporary buffers wherever possible and eliminate duplicate encryption or XOR operations inside loops. These changes reduce execution time, lower memory usage, and improve the scalability of the implementation for large files.

C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication

Each block cipher mode offers different performance and security characteristics. ECB mode is the fastest because each block is encrypted independently, but it is insecure since identical plaintext blocks produce identical ciphertext blocks. CBC mode provides stronger security by chaining blocks together, but encryption cannot be parallelized, leading to higher latency. CFB mode operates as a stream cipher and is suitable for real-time data transmission, although it introduces feedback dependencies that slightly reduce performance. OFB mode also functions as a stream cipher, avoids error propagation, and allows precomputation of the keystream, resulting in lower latency and consistent performance. For real-time secure communication, OFB or CFB is generally more efficient than CBC because they offer lower delay while maintaining good security. ECB should be avoided despite its speed because it does not provide adequate protection for sensitive data.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained

Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation